

CITS5508 Practice Exam — Chapter 8 — ANSWER KEY

Question 1.

(a) (4 marks) Motivations (any two): **speed up training**; enable **2-D/3-D visualization**; sometimes **filter noise**; reduce memory/disk. Drawbacks (any two): **information loss** (may hurt performance); added **pipeline complexity**; transformed features become **harder to interpret**.

(b) (4 marks) The **curse of dimensionality**: high-dimensional space behaves very differently from our 3-D intuition — points are mostly near the boundary and **distances between points become large**. So a high-dimensional training set is **sparse**: instances are far apart, and a new instance is likely far from any training instance, so predictions rely on large **extrapolations**. More dimensions $\Rightarrow$ greater risk of **overfitting**.

(c) (2 marks) The number of instances needed to reach a given density grows **exponentially** with the number of dimensions — e.g. ~ 100 features would need more instances than atoms in the observable universe — so collecting enough data is infeasible.

(d) (2 marks) **Not exactly**. You can apply `inverse_transform` to map back to the original number of dimensions, but the projection **discarded some variance**, so you only recover an **approximation** (close to, but not identical to, the original).

Question 2.

(a) (4 marks) **Projection**: assumes instances lie close to a **lower-dimensional flat subspace** of the high-D space, and projects them perpendicularly onto it. **Manifold learning**: assumes the data lies on a lower-dimensional **manifold** that is **bent/twisted** in the high-D space, and models that manifold (rather than projecting onto a flat subspace).

(b) (4 marks) A **manifold** is a d -dimensional shape bent/twisted inside an n -D space ($d < n$) that **locally resembles a d -D hyperplane** (e.g. the Swiss roll is a 2-D manifold in 3-D). **Manifold learning** = techniques that model this manifold to reduce dimensionality. The **manifold assumption/hypothesis**: most real high-dimensional datasets lie **close to a much lower-dimensional manifold** (often empirically true).

(c) (4 marks) For the Swiss roll, simply dropping a dimension **squashes** the rolled layers on top of each other, destroying the structure. Manifold learning instead **unrolls** the manifold — recovering the intrinsic 2-D layout — so that nearby points on the sheet stay nearby (and the downstream task, e.g. classification, often becomes simpler/linear).

Question 3.

(a) (4 marks) PCA chooses the axis that **preserves the maximum variance** of the projected data. Two equivalent justifications: (1) it **loses the least information** (most variance retained); (2) it **minimises the mean squared distance** between the original points and their projections onto that axis.

(b) (4 marks) The **explained variance ratio** of each principal component is the **proportion of the dataset's total variance** that lies along that component (`explained_variance_ratio_`). To choose the number of dimensions: keep enough components so their **cumulative** explained variance reaches a target (e.g. **95%**), or

look for the **elbow** in the cumulative-variance curve. (For visualization, keep 2-3.)

(c) (4 marks)

```
from sklearn.decomposition import PCA
pca = PCA(n_components=0.95)          # keep 95% of variance
X_reduced = pca.fit_transform(X_train)
print(pca.n_components_)              # number of components actually kept
```

Question 4.

(a) (3 marks) You **cannot say a fixed number** in advance — it depends on the data. PCA keeps the **smallest number of components whose cumulative explained variance $\geq 95\%$** , which could be anywhere from a handful to nearly 1,000 depending on how correlated the features are. (For correlated real-world data it is usually **much less than 1,000**.)

(b) (4 marks) **Reconstruction error** = the mean squared distance between the original data and its **reconstruction** (project down then `inverse_transform` back up). For **anomaly detection**: PCA learns the structure of **normal** data, so normal instances reconstruct with **low** error, whereas **anomalies** (which don't fit that structure) reconstruct with a **much larger** error → flag instances whose reconstruction error exceeds a threshold.

(c) (5 marks) - **Regular PCA** — data fits in memory and isn't enormous. - **Incremental PCA** — data too large for memory, or arriving online (feed mini-batches via `partial_fit`). - **Randomized PCA** — fast approximate PCA when the target dimensionality d is **much smaller than n** ($O(m \cdot d^2 + d^3)$). - **Random projection** — very **high-dimensional** data where even (randomized) PCA is too slow.

Question 5.

(a) (5 marks) **Random projection** multiplies the data by a **random** matrix to map it to a lower-dimensional space. By the **Johnson-Lindenstrauss lemma**, such a random projection **preserves pairwise distances** within a tolerance with high probability, so similar instances stay similar and different ones stay different. The required output dimensionality depends only on the **number of instances m** and the tolerance ϵ — **not on n** (the original number of features).

(b) (4 marks) **LLE step 1**: for each instance, find its k nearest neighbours and the **linear weights** that best reconstruct it from them (capturing local linear relationships). **Step 2**: find a **low-dimensional placement** of all instances that **preserves those local relationships** (weights) as well as possible. Good at unrolling twisted manifolds.

(c) (3 marks) For visualising clusters in 2-D: **t-SNE**. Supervised, projecting onto the most class-discriminative axes: **LDA (Linear Discriminant Analysis)**.